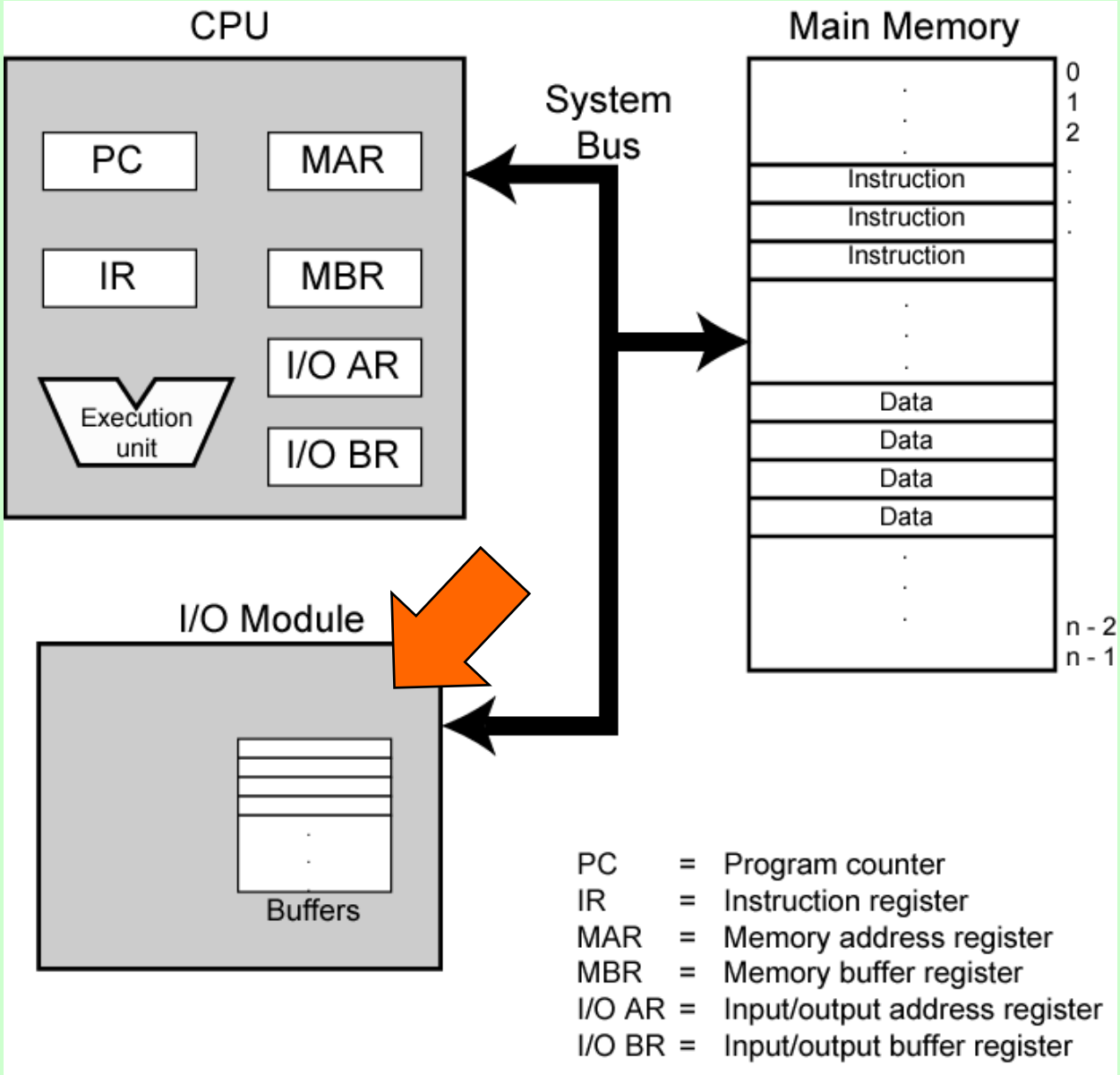
Input/Output

Computer Architecture: Top Level View



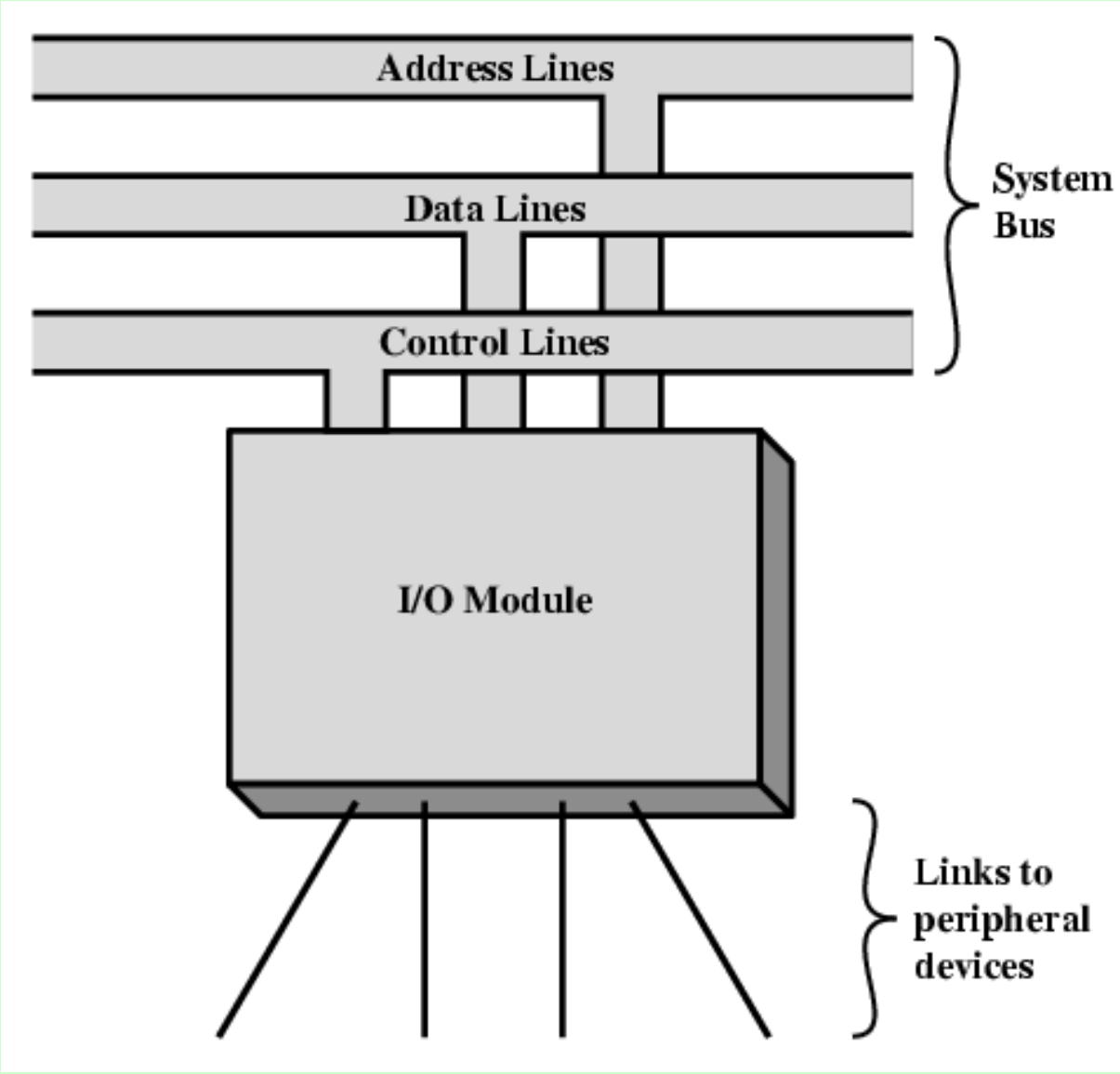
Input/Output Problems

- Wide variety of Peripheral/External Devices
  - Delivering different amounts of data
  - At different speeds
  - In different formats
- All slower than CPU and Memory
- Need I/O modules or Ports

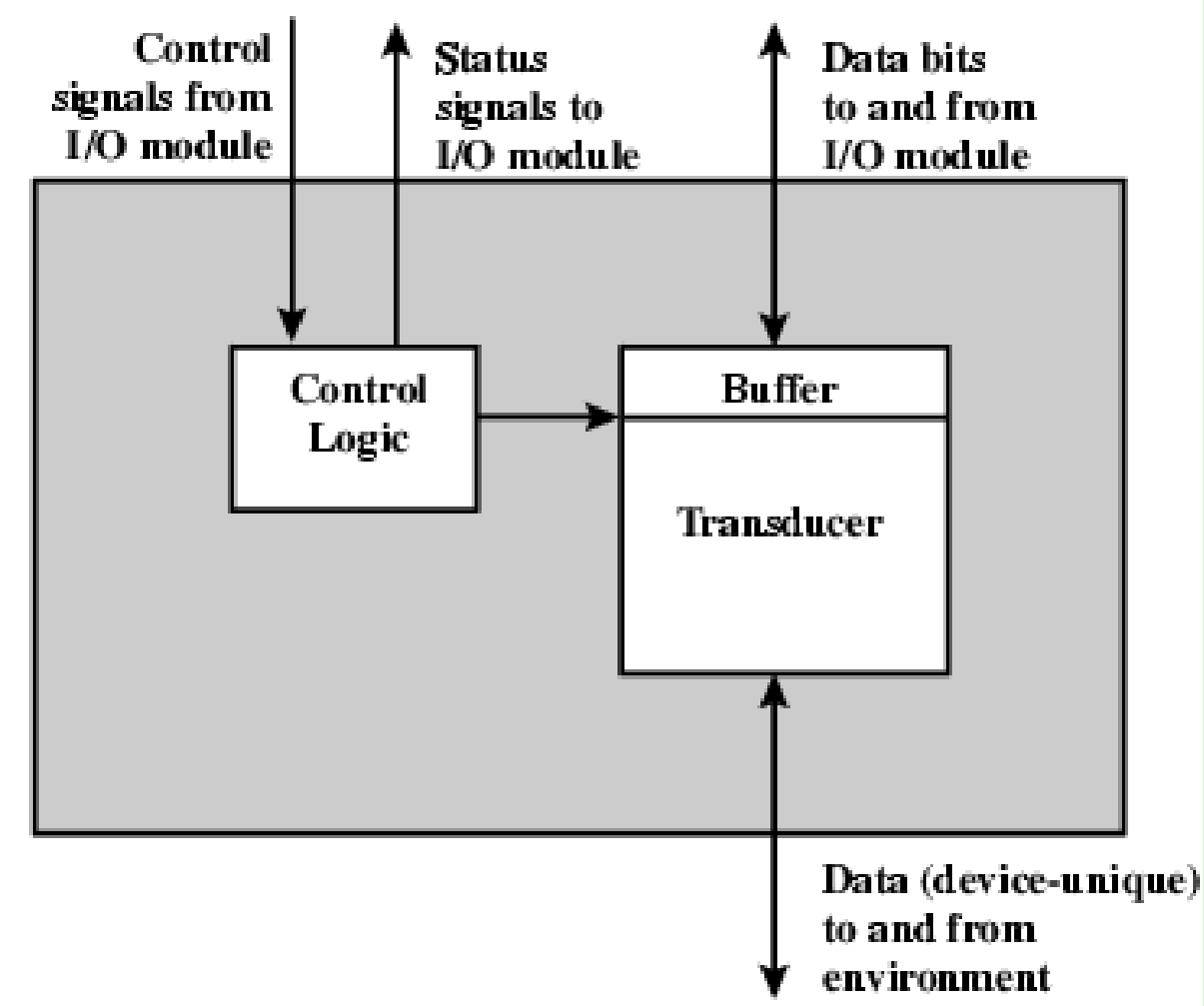
External/Peripheral Devices

- Human interaction
  - Screen, printer, keyboard
- Machine interaction
  - Monitoring and control
- Communication
  - Modem
  - Network Interface Card (NIC)

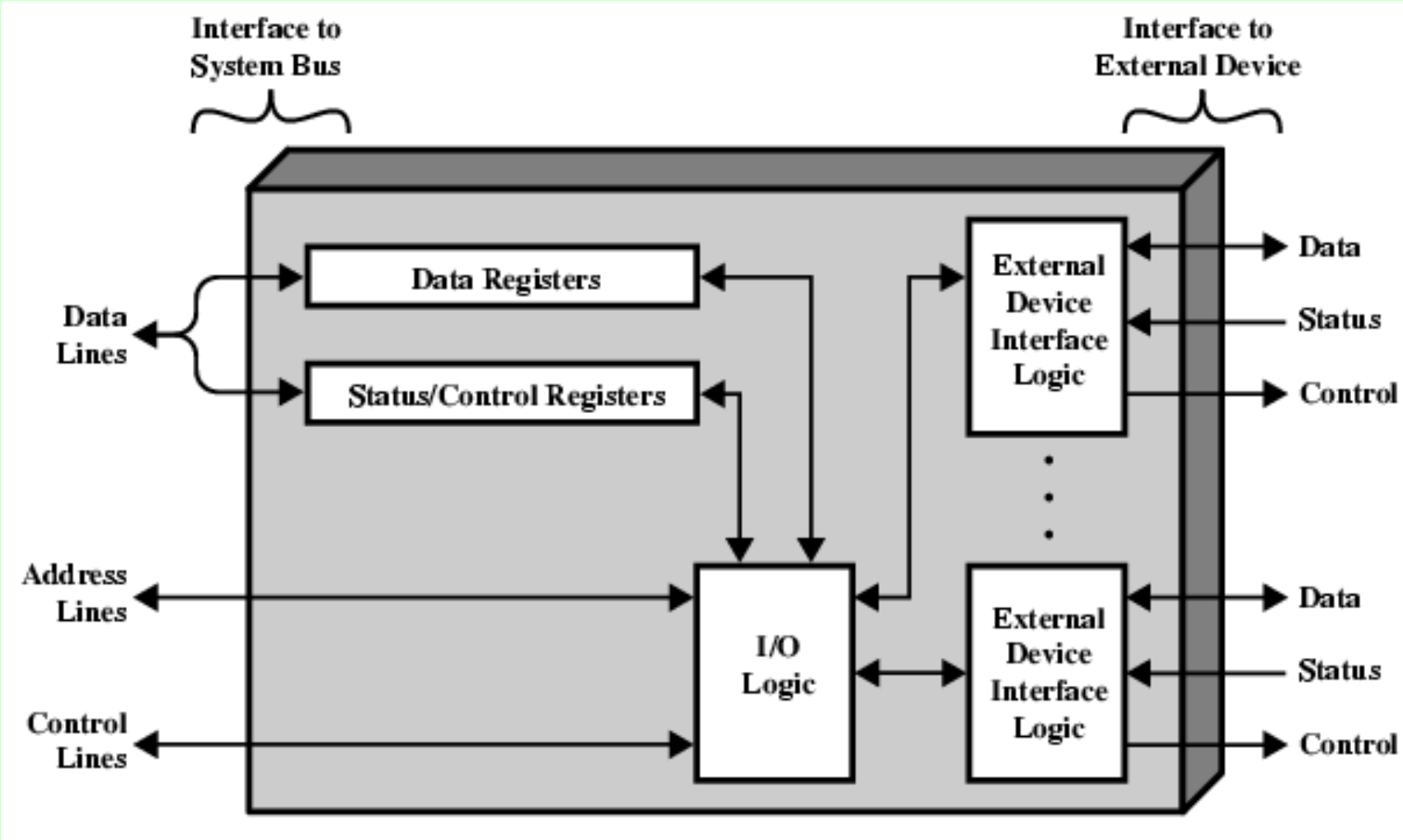
Generic Model of I/O Module



External Device Block Diagram



I/O Module/Port Block Diagram



I/O Module Function

- Control & Timing
- Communication with CPU
- Communication with I/O Device
- Data Buffering
- Error Detection

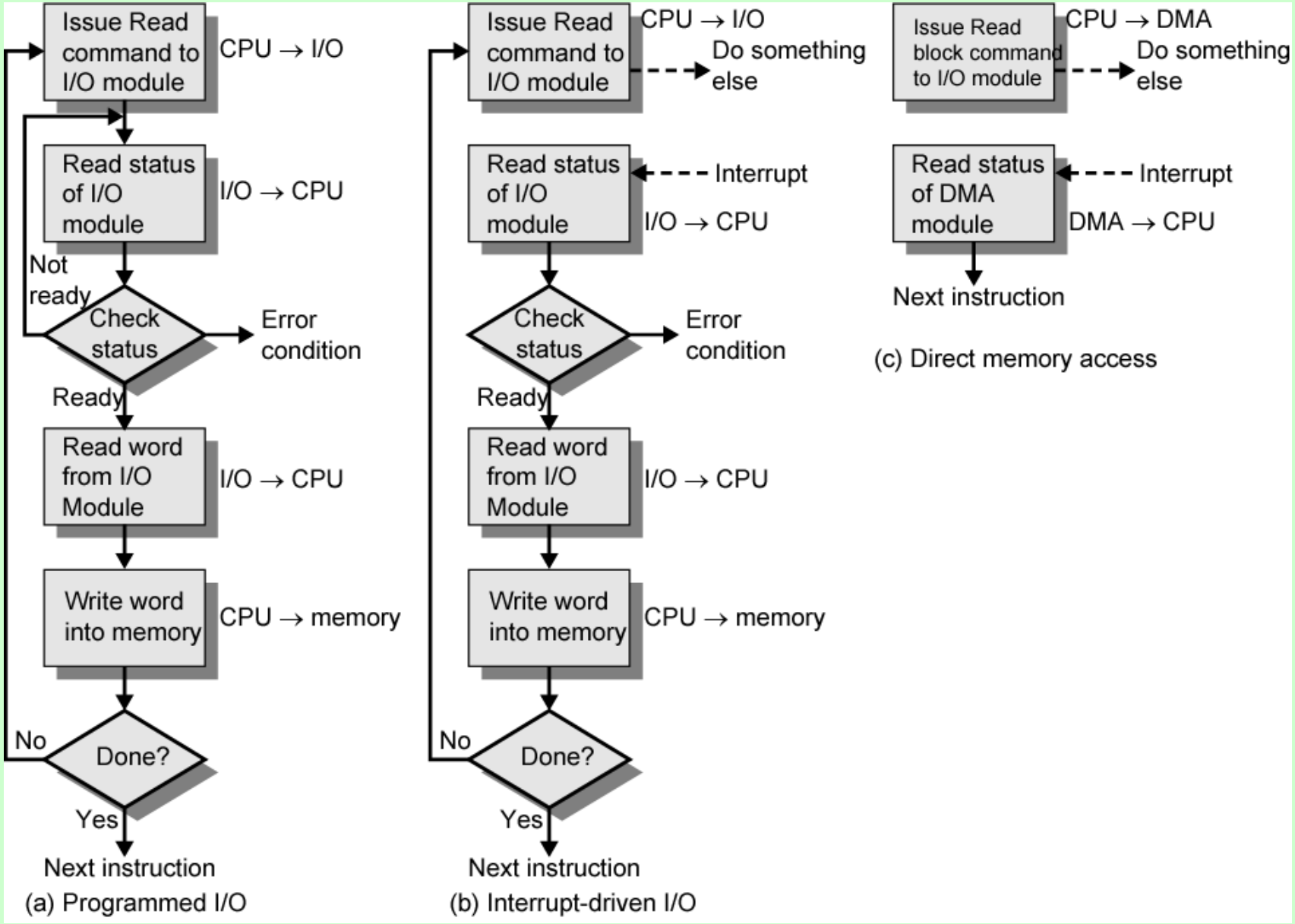
Steps during I/O

- CPU checks I/O module device status
- I/O module returns status to CPU
- If status is ready, CPU requests data transfer
- I/O module gets data from CPU/device in frame/block in buffer
- When buffer is full, I/O module transfers data to CPU (or device)

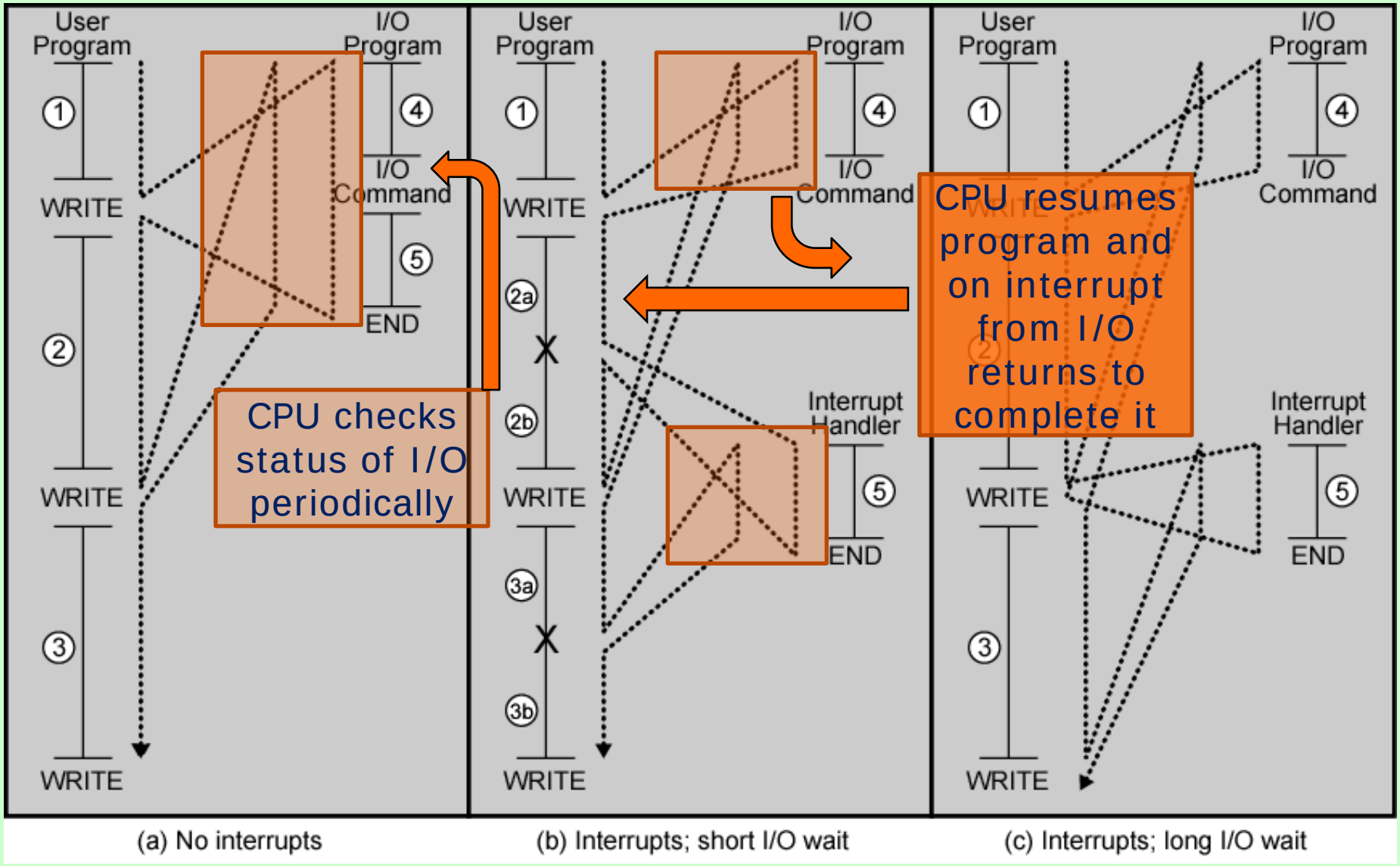
Input Output Techniques

- Programmed
- Interrupt driven
- Direct Memory Access (DMA)

Three Techniques for Input of a Block of Data



Programmed versus Interrupt based I/O



Programmed I/O

- CPU requests I/O operation
- I/O module performs operation
- I/O module sets status bits
- CPU checks status bits periodically
- I/O module does not inform CPU directly or interrupt CPU
- CPU waits for I/O module to complete operation, wastes CPU time

I/O Commands

- CPU issues command
  - Control - telling module what to do
    - e.g. spin up disk
  - Test - check status
    - e.g. power? Error?
  - Read/Write
    - Module transfers data via buffer from/to device

- CPU command also has ID
  - Identifies (ID) I/O module (& device if >1 per module)

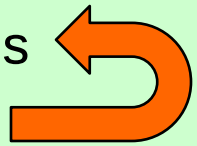
Addressing I/O Devices

- I/O data transfer is very like memory access (CPU viewpoint)
- Each device given either
  - —unique identifier
  - —address from memory space
- I/O commands contain identifier (address) as operand

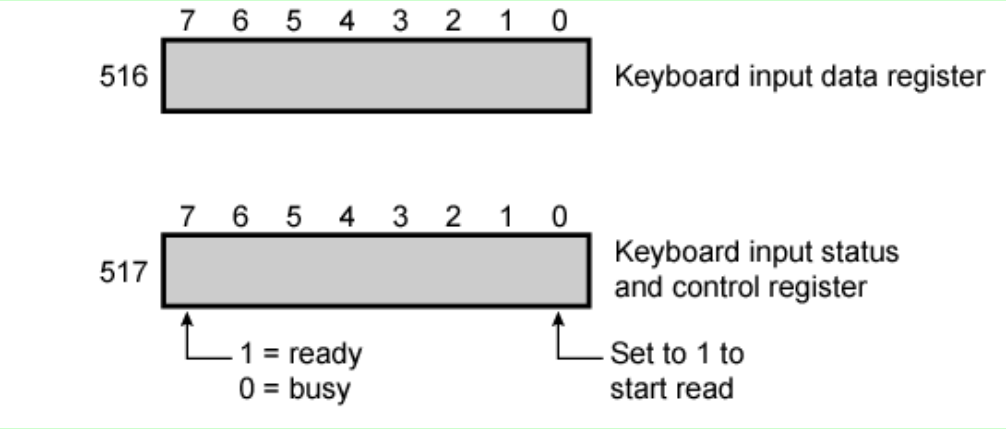
I/O Mapping

- Memory mapped I/O
  - Devices and memory share an address space
  - I/O looks just like memory read/write
  - No special commands for I/O

- Isolated I/O
  - Separate address spaces
  - Need I/O or memory select lines
  - Special commands for I/O
    - Limited set



Memory Mapped and Isolated I/O



ADDRESS	INSTRUCTION	OPERAND	COMMENT
200	Load AC	"1"	Load accumulator
	Store AC	517	Initiate keyboard read
202	Load AC	517	Get status byte
	Branch if Sign = 0	202	Loop until ready
	Load AC	516	Load data byte

(a) Memory-mapped I/O

ADDRESS	INSTRUCTION	OPERAND	COMMENT
200	Load I/O	5	Initiate keyboard read
201	Test I/O	5	Check for completion
	Branch Not Ready	201	Loop until complete
	In	5	Load data byte

(b) Isolated I/O

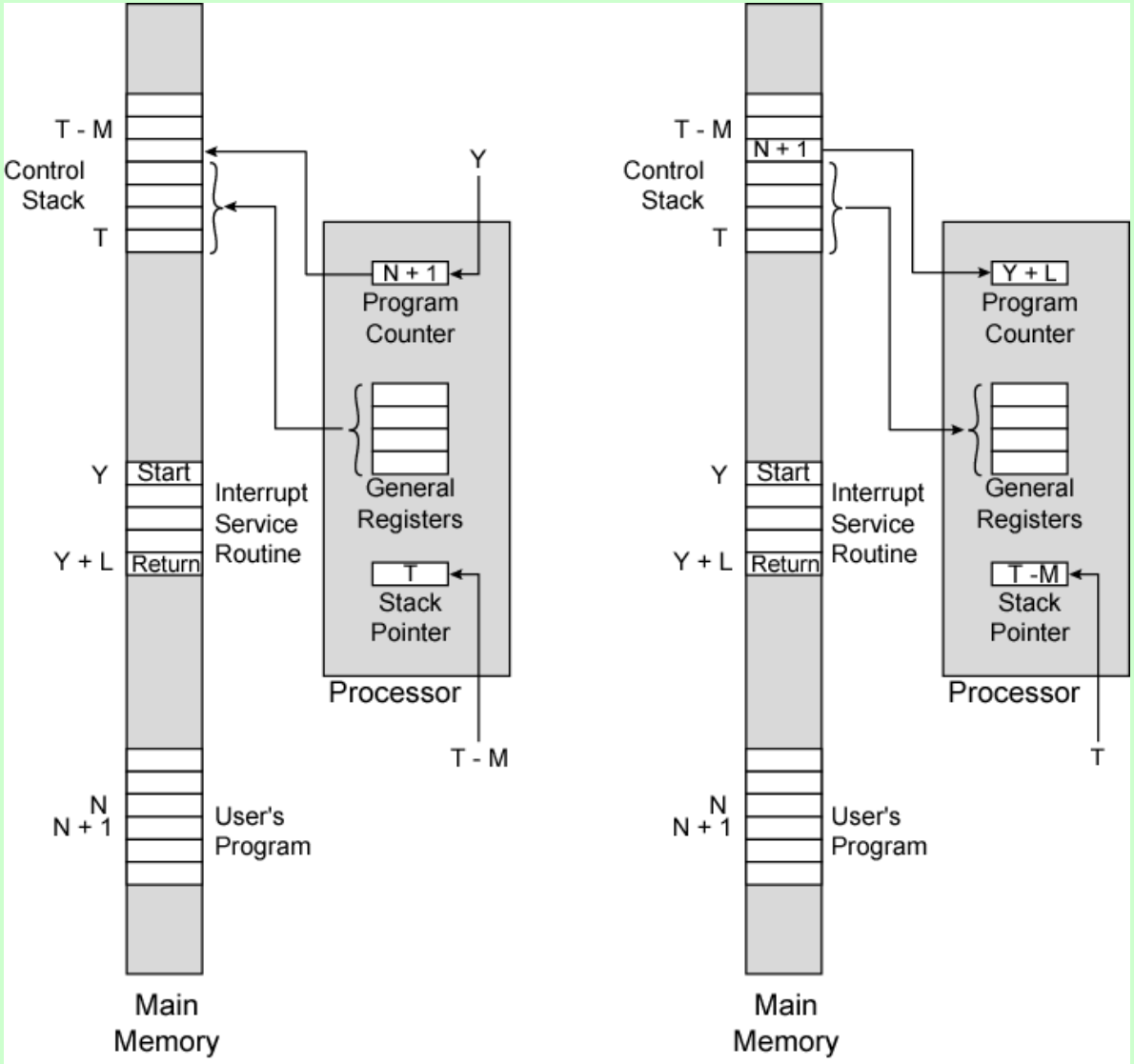
Interrupt Driven I/O

- Overcomes CPU waiting
- No repeated CPU checking of device
- I/O module interrupts when ready

Interrupt Driven I/O (CPU Viewpoint)

- Issue I/O command
- Does other work
- Check for interrupt at end of each instruction cycle
- If interrupted (I/O module/device ready):-
  - Save context (registers/variables)
  - Process interrupt
    - Transfer data to/from Memory to I/O

Changes in Memory and Registers for an Interrupt



Design Issues: Interrupt based I/O

- How do you identify the module issuing the interrupt?
- How do you deal with multiple interrupts?
  - i.e. an interrupt handler being interrupted

Identifying Interrupting Module

- Poll
  - CPU asks each module in turn
  - Slow
- Daisy Chain
  - Interrupt Acknowledge sent down a chain
  - Module responsible places vector on bus
  - CPU uses vector to identify handler routine
- Bus Master
  - Module must claim the bus before it can raise interrupt
  - e.g. PCI & SCSI

Multiple Interrupts

- Each interrupt line has a priority
- Higher priority lines can interrupt lower priority lines
- If bus mastering only current master can interrupt

Direct Memory Access

- Interrupt driven and programmed I/O require active CPU intervention
  - Transfer rate is limited
  - CPU is tied up
- DMA is the answer
  - DMA controller takes over from CPU for I/O

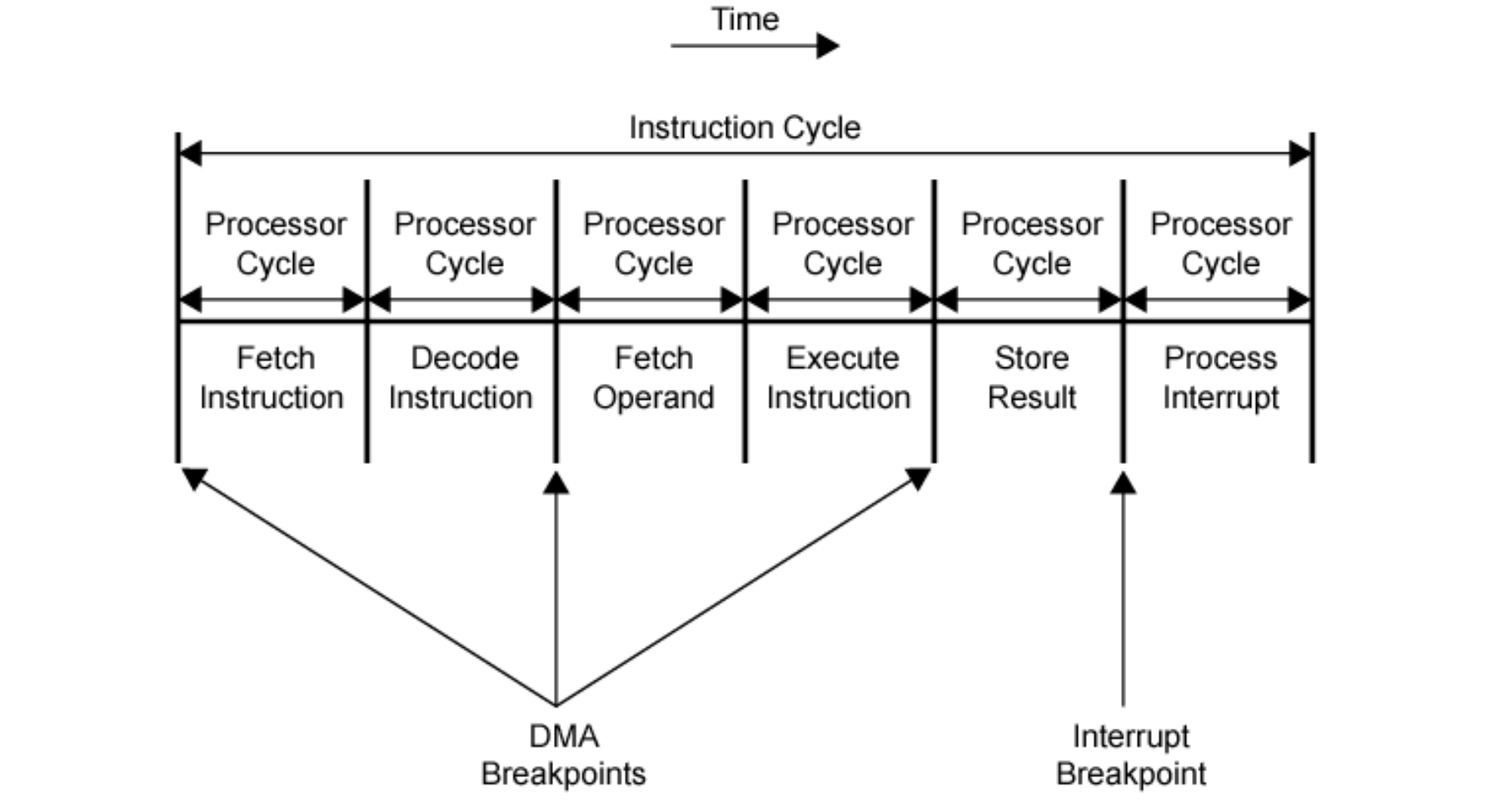
DMA Operation

- CPU tells DMA controller:-
  - Read/Write
  - Device address
  - Starting address of memory block for data
  - Amount of data to be transferred
- CPU carries on with other work
- DMA controller deals with transfer
- DMA controller sends interrupt when finished

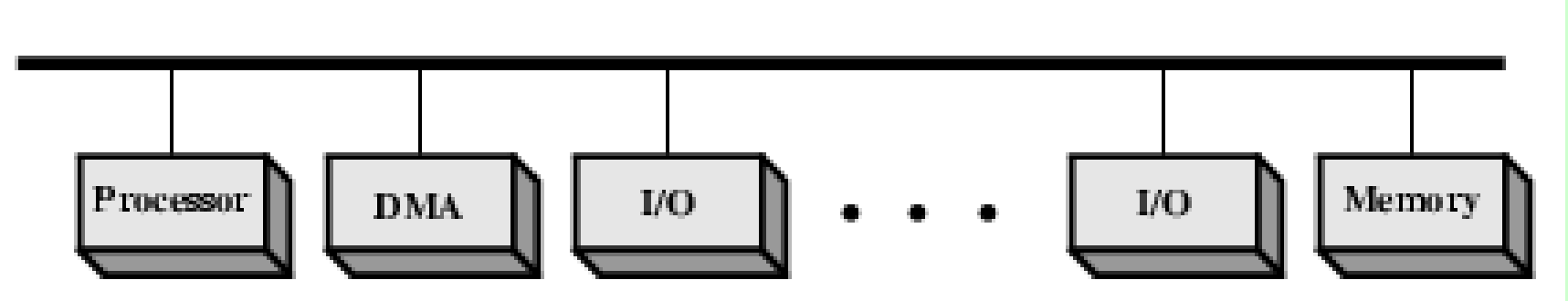
DMA Transfer
Cycle Stealing

- DMA controller takes over bus for a cycle
- Transfer of one word of data
- Not an interrupt
  - CPU does not switch context
- CPU suspended just before it accesses bus
  - i.e. before an operand or data fetch or a data write
- Slows down CPU but not as much as CPU doing transfer

DMA and Interrupt Breakpoints During an Instruction Cycle

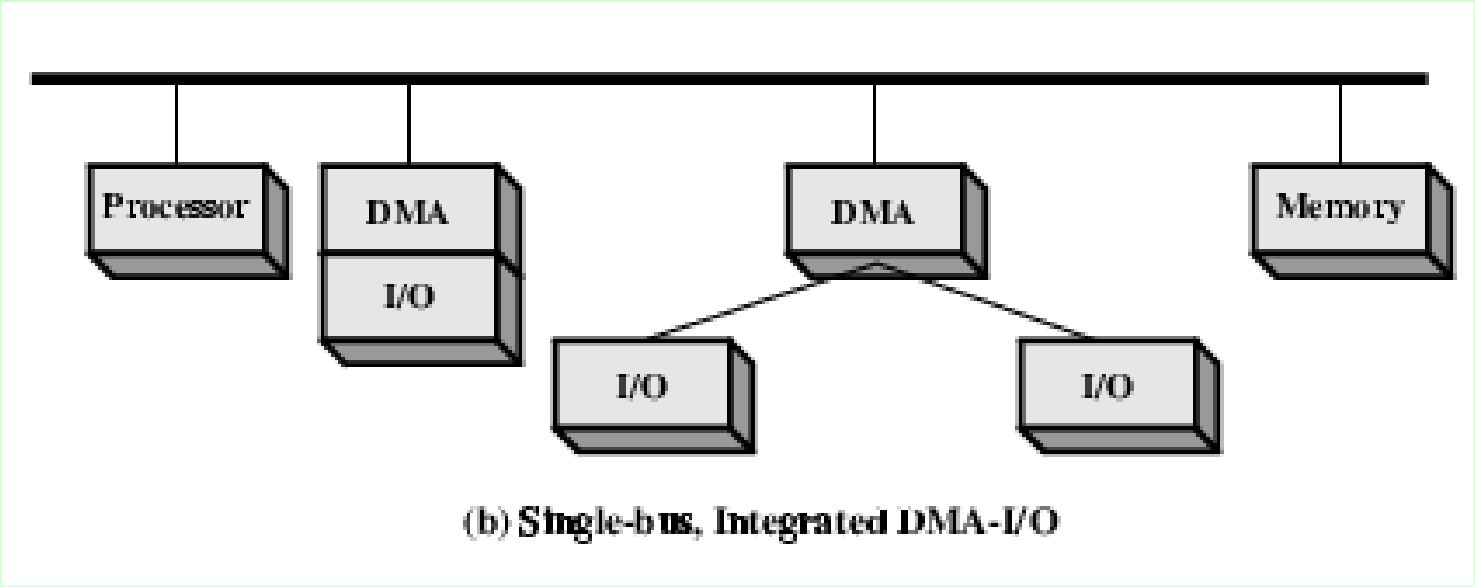


DMA Configurations (1)



- Single Bus, Detached DMA controller
- Each transfer uses bus twice
  - I/O to DMA then DMA to memory
- CPU is suspended twice

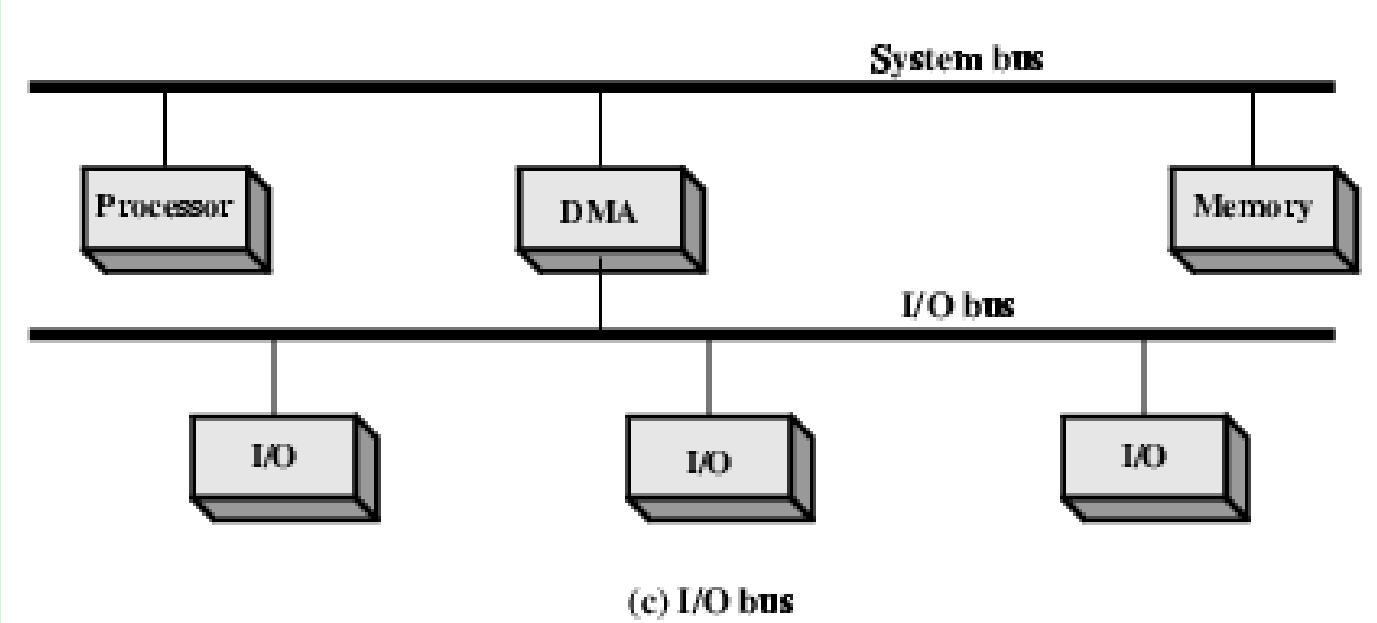
DMA Configurations (2)



(b) Single-bus, Integrated DMA-I/O

- Single Bus, Integrated DMA controller
- Controller may support >1 device
- Each transfer uses bus once
  - —DMA to memory
- CPU is suspended once

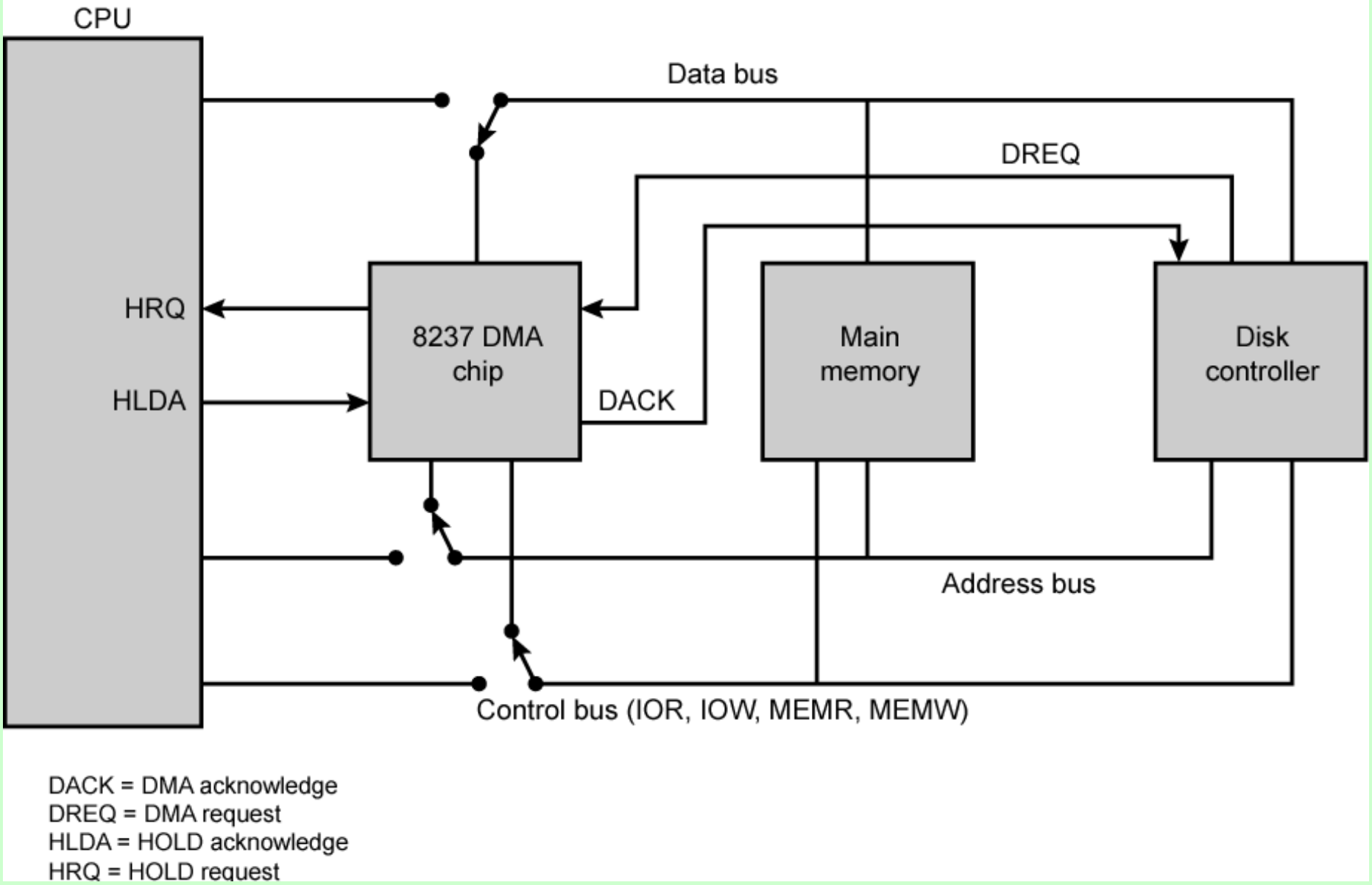
DMA Configurations (3)



(c) I/O bus

- Separate I/O Bus
- Bus supports all DMA enabled devices
- Each transfer uses bus once
  - —DMA to memory
- CPU is suspended once

8237 DMA Usage of Systems Bus



DACK = DMA acknowledge
DREQ = DMA request
HLDA = HOLD acknowledge
HRQ = HOLD request

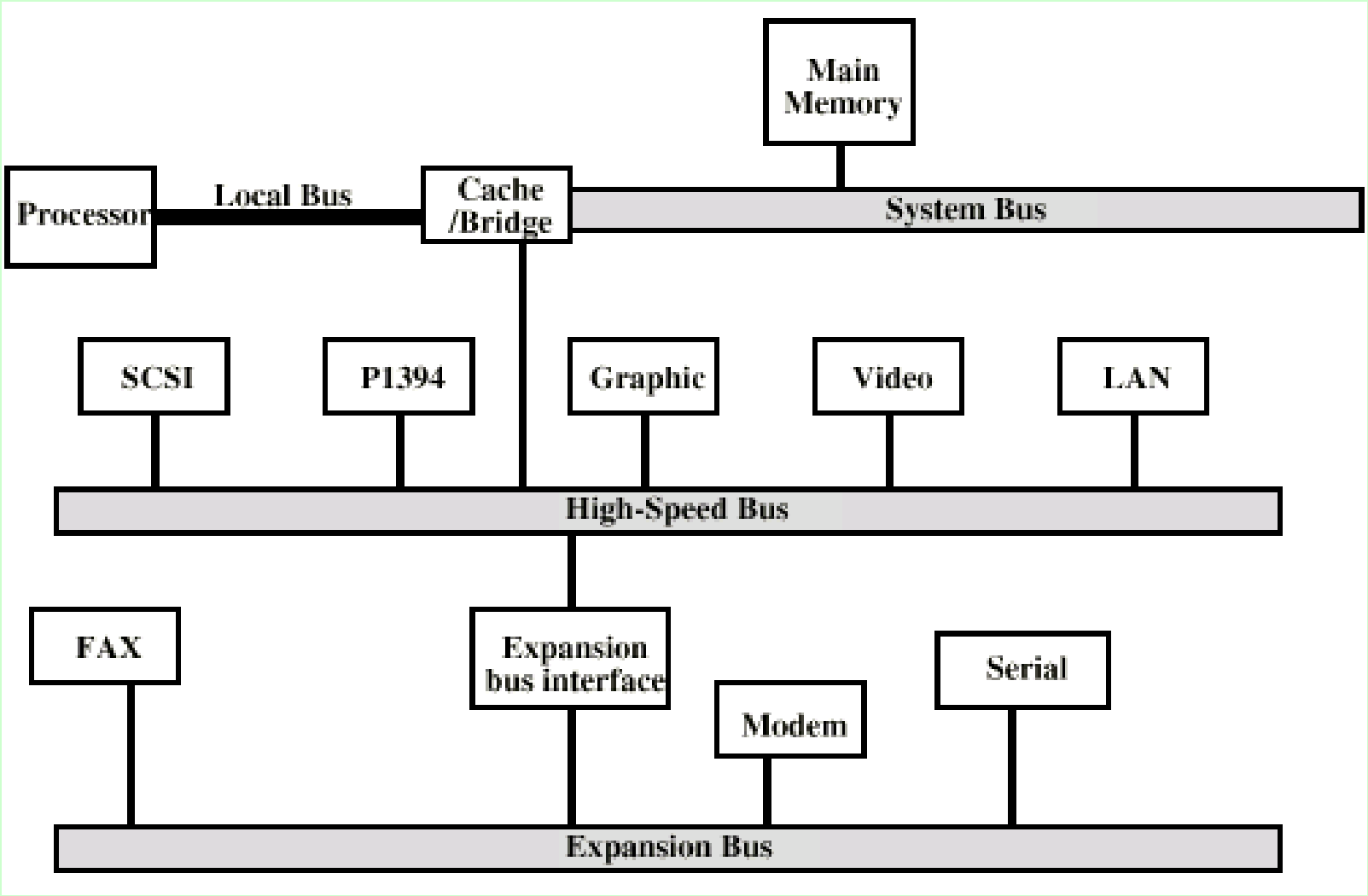
Big and Yellow?

- What do buses look like?
  - Parallel lines on circuit boards
  - Ribbon cables
  - Strip connectors on mother boards
    - e.g. PCI
  - Sets of wires

Single Bus Problems

- Lots of devices on one bus leads to:
  - Propagation delays
    - Long data paths mean that co-ordination of bus use can adversely affect performance
    - If aggregate data transfer approaches bus capacity
- Most systems use multiple buses to overcome these problems

High Performance Bus



Bus Arbitration

- More than one module controlling the bus
- e.g. CPU and DMA controller
- Only one module may control bus at one time
- Arbitration may be centralised or distributed

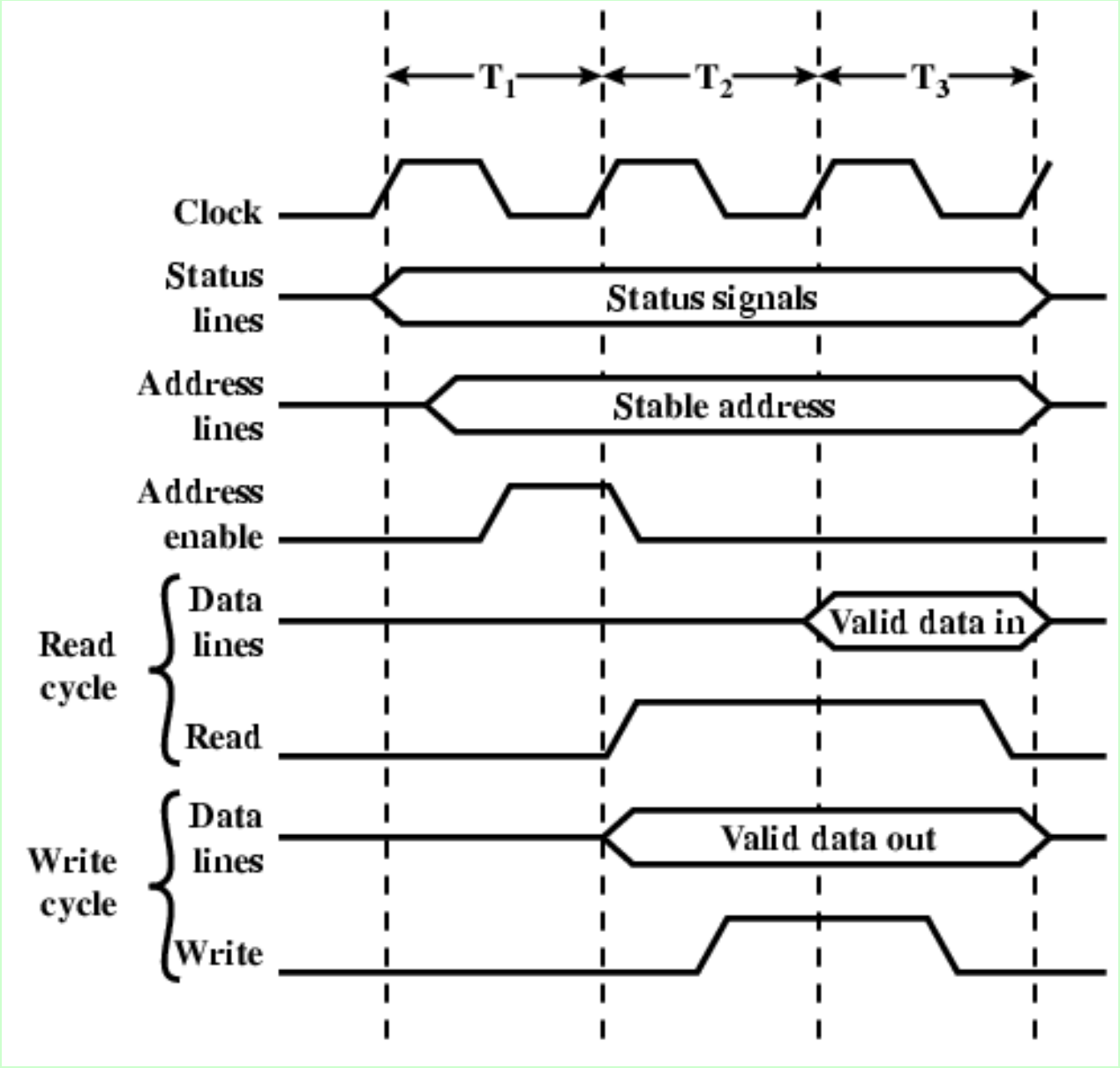
Centralised or Distributed Arbitration

- Centralised
  - Single hardware device controlling bus access
    - Bus Controller
    - Arbiter
  - May be part of CPU or separate
- Distributed
  - Each module may claim the bus
  - Control logic on all modules

Bus Timing

- Co-ordination of events on bus
- Synchronous
  - Events determined by clock signals
  - Control Bus includes clock line
  - A single 1-0 is a bus cycle
  - All devices can read clock line
  - Usually sync on leading edge
  - Usually a single cycle for an event

Synchronous Timing Diagram



Assignment-1 (Sat 17 Apr by 1500)

- Check out Universal Serial Bus (USB) 3.0 versus 2.0 Timing & Protocol
- Compare with other serial communication standards e.g. Ethernet, Broadband Modem
- Max 2 Pages A-4, Handwritten